

04 - zz - IP Fundamentals - EXCERCISE (my solution)

Lesson 2 Exercises: IP Fundamentals & Basic Connectivity

Complete these exercises to build your IP addressing and Ansible skills.

Exercise 1: Deploy and Verify

Objective: Deploy the lab topology, apply configuration with Ansible, and verify connectivity.

Steps

1. Deploy the topology:

```
cd lessons/clab/02-ip-fundamentals  
containerlab deploy -t topology/lab.clab.yml
```

[*]-[kc-debian13-test00-

Klon]-[/kubecraft/DrewElliot_kubecraft/lessons/clab/02-ip-

fundamentals/topology]

└─> clab deploy

20:01:42 INFO Containerlab started version=0.75.0

20:01:42 INFO Parsing & checking topology file=lab.clab.yml

20:01:42 INFO Creating docker network name=clab IPv4 subnet=172.20.20.0/24

IPv6 subnet=3fff:172:20:20::/64 MTU=0

20:01:42 INFO Pulling image image=docker.io/library/alpine:3.20

3.20: Pulling from library/alpine

25f1d6b1951a: Pull complete

d4050d56ebf2: Download complete

3a030ca3f633: Download complete

Digest:

sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2a4b6bc

Status: Downloaded newer image for alpine:3.20

20:01:44 INFO Done pulling image image=docker.io/library/alpine:3.20

20:01:44 INFO Creating lab directory

path=/kubecraft/DrewElliot_kubecraft/lessons/clab/02-ip-

fundamentals/topology/clab-ip-fundamentals

20:01:44 INFO Creating container name=srl2

20:01:44 INFO Creating container name=srl1

20:01:45 INFO Created link: srl1:e1-2 ──── srl2:e1-1

20:01:45 INFO Running postdeploy actions kind=srl node=srl1

20:01:45 INFO Running postdeploy actions kind=srl node=srl2

20:02:15 INFO Creating container name=host2

20:02:15 INFO Created link: srl2:e1-2 ──── host2:eth1

20:02:16 INFO Creating container name=host1

20:02:16 INFO Created link: host1:eth1 ──── srl1:e1-1

20:02:19 INFO Executed command node=host2 command="ip link set eth1 up"

stdout=""

20:02:19 INFO Executed command node=host2 command="ip addr add 10.1.3.2/24

dev eth1" stdout=""

20:02:19 INFO Failed to execute command command="ip route add default via

10.1.3.1 dev eth1" node=host2 rc=2 stdout=""

stderr=

| ip: RTNETLINK answers: File exists

20:02:19 INFO Executed command node=host1 command="ip link set eth1 up"

stdout=""

20:02:19 INFO Executed command node=host1 command="ip addr add 10.1.1.2/24

dev eth1" stdout=""

20:02:19 INFO Failed to execute command command="ip route add default via

10.1.1.1 dev eth1" node=host1 rc=2 stdout=""

stderr=

| ip: RTNETLINK answers: File exists

```
20:02:19 INFO Adding host entries path=/etc/hosts
```

```
20:02:19 INFO Adding SSH config for nodes path=/etc/ssh/ssh_config.d/clab-  
ip-fundamentals.conf
```

Name	Kind/Image	State
IPv4/6 Address		
clab-ip-fundamentals-host1	linux	running
172.20.20.3		
	alpine:3.20	
3fff:172:20:20::5		
clab-ip-fundamentals-host2	linux	running
172.20.20.2		
	alpine:3.20	
3fff:172:20:20::4		
clab-ip-fundamentals-srl1	srl	running
172.20.20.11		
	ghcr.io/nokia/srlinux:24.10.1	
3fff:172:20:20::3		
clab-ip-fundamentals-srl2	srl	running
172.20.20.12		
	ghcr.io/nokia/srlinux:24.10.1	
3fff:172:20:20::2		

2. Apply router configuration with Ansible:

```
cd ansible
```

```
ansible-playbook -i inventory.yml playbook.yml
```

```

[*]--[kc-debian13-test00-
Klon]--[ /kubecraft/DrewElliot_kubecraft/lessons/clab/02-ip-
fundamentals/ansible]
└─> cd /kubecraft/DrewElliot_kubecraft/lessons/clab/02-ip-
fundamentals/ansible
]
[*]--[kc-debian13-test00-
Klon]--[ /kubecraft/DrewElliot_kubecraft/lessons/clab/02-ip-
fundamentals/ansible]
└─> ansible-playbook -i inventory.yml playbook.yml
]
PLAY [Configure SR Linux router interfaces]
*****
*****
]
TASK [Apply interface configuration via JSON-RPC]
*****
*****
ok: [srl1]
ok: [srl2]
]
TASK [Show configuration result]
*****
*****
ok: [srl1] => {
  "msg": "srl1: Configuration applied successfully"
}
ok: [srl2] => {
  "msg": "srl2: Configuration applied successfully"
}
]
TASK [Verify interfaces are configured]
*****
*****
ok: [srl1]
ok: [srl2]
]
TASK [Display configured interfaces]
*****
*****
ok: [srl1] => {
  "msg": "srl1 interfaces: [{ 'interface': [{ 'name': 'ethernet-1/1.0',
'oper-state': 'up', 'index': '2'}, { 'name': 'ethernet-1/2.0', 'oper-state':
'up', 'index': '
3' } ] } ] }"
}
ok: [srl2] => {

```

```

"msg": "srl2 interfaces: [{ 'interface': [{ 'name': 'ethernet-1/1.0',
'oper-state': 'up', 'index': '2'}, { 'name': 'ethernet-1/2.0', 'oper-state':
'up', 'index': '
3'}]]]"
}

```

PLAY RECAP

```

*****
*****
*
srl1                : ok=4    changed=0    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
srl2                : ok=4    changed=0    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0

```

3. Verify host-to-router connectivity:

```

docker exec clab-ip-fundamentals-host1 ping -c 3 10.1.1.1
docker exec clab-ip-fundamentals-host2 ping -c 3 10.1.3.1

```

```

[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/02-ip-fundamentals]
└─> docker exec clab-ip-fundamentals-host1 ping -c 3 10.1.1.1
PING 10.1.1.1 (10.1.1.1): 56 data bytes
64 bytes from 10.1.1.1: seq=0 ttl=64 time=3.543 ms
64 bytes from 10.1.1.1: seq=1 ttl=64 time=1.441 ms
64 bytes from 10.1.1.1: seq=2 ttl=64 time=2.306 ms
--- 10.1.1.1 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 1.441/2.430/3.543 ms
[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/02-ip-fundamentals]
└─> docker exec clab-ip-fundamentals-host2 ping -c 3 10.1.3.1
PING 10.1.3.1 (10.1.3.1): 56 data bytes
64 bytes from 10.1.3.1: seq=0 ttl=64 time=4.738 ms
64 bytes from 10.1.3.1: seq=1 ttl=64 time=2.133 ms
64 bytes from 10.1.3.1: seq=2 ttl=64 time=1.849 ms
--- 10.1.3.1 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 1.849/2.906/4.738 ms

```

4. Verify router-to-router connectivity:

```
docker exec -it clab-ip-fundamentals-srl1 sr_cli
```

Inside SR Linux:

```
ping 10.1.2.2 network-instance default
exit
```

```
[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/02-ip-fundamentals]
└─> docker exec -it clab-ip-fundamentals-srl1 sr_cli
Using configuration file(s): []
Welcome to the srlinux CLI.
Type 'help' (and press <ENTER>) if you need any help using this.
--{ + running }--[ ]--
A:srl1# ping 10.1.2.2 network-instance default
Using network instance default
PING 10.1.2.2 (10.1.2.2) 56(84) bytes of data.
64 bytes from 10.1.2.2: icmp_seq=1 ttl=64 time=104 ms
64 bytes from 10.1.2.2: icmp_seq=2 ttl=64 time=3.17 ms
64 bytes from 10.1.2.2: icmp_seq=3 ttl=64 time=3.36 ms
64 bytes from 10.1.2.2: icmp_seq=4 ttl=64 time=4.19 ms
64 bytes from 10.1.2.2: icmp_seq=5 ttl=64 time=3.14 ms
^CCommand execution aborted : 'ping 10.1.2.2 network-instance default'
--{ + running }--[ ]--
A:srl1#
EOF encountered
```

5. Try cross-subnet ping (this should fail):

```
docker exec clab-ip-fundamentals-host1 ping -c 3 10.1.3.2
```

```
[x]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/02-ip-fundamentals]
└─> docker exec clab-ip-fundamentals-host1 ping -c 3 10.1.3.2
PING 10.1.3.2 (10.1.3.2): 56 data bytes
^C

```

Failed - no ping possible

6. Inspect ARP tables:

```
# On host1
```

```
docker exec clab-ip-fundamentals-host1 arp -n
```

```
# On srl1
```

```
docker exec -it clab-ip-fundamentals-srl1 sr_cli -c "show arpnd arp-  
entries"
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/02-ip-fundamentals]
```

```
└─> docker exec clab-ip-fundamentals-host1 arp -n
```

```
? (172.20.20.1) at 2a:6f:71:25:2e:0c [ether] on eth0
```

```
? (10.1.1.1) at 1a:9d:02:ff:00:01 [ether] on eth1
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/02-ip-fundamentals]
```

```
└─> docker exec -it clab-ip-fundamentals-srl1 sr_cli -c "show arpnd arp-  
entries"
```

```
+-----+-----+-----+-----+  
+-----+-----+-----+-----+  
-----+
```

Interface	Subinterface	Neighbor	Origin
Link layer address			Expiry

```
|
```

```
+=====+=====+=====+=====+  
=====+=====+=====+=====+  
=====+
```

ethernet-1/1	0	10.1.1.2	dynamic
AA:C1:AB:B0:39:EA			3 hours from now

```
|
```

ethernet-1/2	0	10.1.2.2	dynamic
1A:B0:03:FF:00:01			3 hours from now

```
|
```

mgmt0	0	172.20.20.1	dynamic
2A:6F:71:25:2E:0C			3 hours from now

```
|
```

```
+-----+-----+-----+-----+  
+-----+-----+-----+-----+  
-----+
```

```
-----  
-----  
-----
```

```
Total entries : 3 (0 static, 3 dynamic)
```

```
-----  
-----  
-----
```

```
Parsing error: Unknown token 'commit'. Options are ['!', '#', '/', 'acl',  
'back', 'bash', 'bfd', 'cls', 'date', 'diff', 'echo', 'enter',  
'environment', 'exit', 'file', 'help', 'histo
```

```
ry', 'info', 'interface', 'list', 'monitor', 'network-instance', 'passwd',  
'ping', 'ping6', 'platform', 'pwc', 'qos', 'quit', 'references', 'routing-  
policy', 'save', 'show', 'source'
```

```
, 'ssh', 'system', 'tcptraceroute', 'tech-support', 'tools', 'traceroute',  
'traceroute6', 'tree', 'tunnel', 'tunnel-interface', 'watch', '{']
```

Deliverables

Create a file `exercises/exercise1-results.md` with:

- Which pings succeeded and which failed
 - *See this file*
 - The ARP table entries you observed
 - *See this file*
 - Your explanation of why cross-subnet ping fails
 - *Because routing on the router devices is not proper configured (see next chapter ;-)*
-

Exercise 2: Read the Config

Objective: Understand how Jinja2 templates generate device-specific configurations.

Steps

1. Open the Jinja2 template:

```
cat ansible/templates/srl_interfaces.json.j2
```

```

{# Generates SR Linux CLI commands for interface configuration.
  Variables come from host_vars/<hostname>.yaml:
  interfaces:
    - name: ethernet-1/1
      ipv4_address: 10.1.1.1/24
      description: Link to host1
#}
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "cli",
  "params": {
    "commands": [
      "enter candidate",
      {% for iface in interfaces %}
        "set / interface {{ iface.name }} admin-state enable",
        "set / interface {{ iface.name }} subinterface 0 admin-state enable",
        "set / interface {{ iface.name }} subinterface 0 description '{{
iface.description }}'",
        "set / interface {{ iface.name }} subinterface 0 ipv4 admin-state
enable",
        "set / interface {{ iface.name }} subinterface 0 ipv4 address {{
iface.ipv4_address }}",
        "set / network-instance default interface {{ iface.name }}.0",
      {% endfor %}
      "commit now"
    ]
  }
}

```

2. Open srl2's host variables:

```
cat ansible/host_vars/srl2.yaml
```

```

---
interfaces:
  - name: ethernet-1/1
    ipv4_address: 10.1.2.2/24
    description: Link to srl1
  - name: ethernet-1/2
    ipv4_address: 10.1.3.1/24
    description: Link to host2

```

3. Manually trace through the template, substituting srl2's variables. For each `{{ iface.name }}` and `{{ iface.ipv4_address }}`, write out what the actual value would be.
4. Write out the complete list of CLI commands that would be generated for srl2.

Stolen from "solutions"

```
enter candidate
set / interface ethernet-1/1 admin-state enable
set / interface ethernet-1/1 subinterface 0 admin-state enable
set / interface ethernet-1/1 subinterface 0 description 'Link to srl1'
set / interface ethernet-1/1 subinterface 0 ipv4 admin-state enable
set / interface ethernet-1/1 subinterface 0 ipv4 address 10.1.2.2/24
set / network-instance default interface ethernet-1/1.0
set / interface ethernet-1/2 admin-state enable
set / interface ethernet-1/2 subinterface 0 admin-state enable
set / interface ethernet-1/2 subinterface 0 description 'Link to host2'
set / interface ethernet-1/2 subinterface 0 ipv4 admin-state enable
set / interface ethernet-1/2 subinterface 0 ipv4 address 10.1.3.1/24
set / network-instance default interface ethernet-1/2.0
commit now
```

The template loops twice (once per interface in srl2's `host_vars`), generating the full set of CLI commands. The same template with srl1's variables would produce different commands with different IPs -- that's the value of templating.

Deliverables

Create a file `exercises/srl2-commands.txt` with the exact CLI commands generated when the template is rendered for srl2.

Exercise 3: Add a Loopback Interface

Objective: Modify the Ansible configuration to add a new interface.

Steps

1. Edit `ansible/host_vars/srl1.yml` and add a loopback interface:

```
- name: lo0
  ipv4_address: 10.10.10.1/32
  description: Loopback for management
```

`host_vars/srl1.yml`

```
---
interfaces:
  - name: ethernet-1/1
    ipv4_address: 10.1.1.1/24
    description: Link to host1
  - name: ethernet-1/2
    ipv4_address: 10.1.2.1/24
    description: Link to srl2
  - name: lo0
    ipv4_address: 10.10.10.1/32
    description: Loopback for management
```

1. Re-run the playbook:

```
cd ansible
ansible-playbook -i inventory.yml playbook.yml
```

`ansible-playbook -i inventory.yml playbook.yml`

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-
```

```
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
```

```
└─> ansible-playbook -i inventory.yml playbook.yml
```

```
PLAY [Configure SR Linux router interfaces]
```

```
*****
```

```
*****
```

```
*****
```

```
TASK [Apply interface configuration via JSON-RPC]
```

```
*****
```

```
*****
```

```
*****
```

```
ok: [srl2]
```

```
ok: [srl1]
```

```
TASK [Show configuration result]
```

```
*****
```

```
*****
```

```
*****
```

```
ok: [srl1] => {
```

```
  "msg": "srl1: Configuration applied successfully"
```

```
}
```

```
ok: [srl2] => {
```

```
  "msg": "srl2: Configuration applied successfully"
```

```
}
```

```
TASK [Verify interfaces are configured]
```

```
*****
```

```
*****
```

```
*****
```

```
ok: [srl1]
```

```
ok: [srl2]
```

```
TASK [Display configured interfaces]
```

```
*****
```

```
*****
```

```
*****
```

```
ok: [srl1] => {
```

```
  "msg": "srl1 interfaces: [{ 'interface': [{ 'name': 'ethernet-1/1.0',
```

```
    'oper-state': 'up', 'index': '2'}, { 'name': 'ethernet-1/2.0', 'oper-state':
```

```
    'up', 'index': '3'} ]}]"
```

```
}
```

```
ok: [srl2] => {
```

```
  "msg": "srl2 interfaces: [{ 'interface': [{ 'name': 'ethernet-1/1.0',
```

```
    'oper-state': 'up', 'index': '2'}, { 'name': 'ethernet-1/2.0', 'oper-state':
```

```
    'up', 'index': '3'} ]}]"
```

```
}
```

```
PLAY RECAP
```

```
*****
```

```
*****
```

```
*****
```

```
srl1 : ok=4 changed=0 unreachable=0 failed=0
```

```
skipped=0 rescued=0 ignored=0
```

```
srl2 : ok=4 changed=0 unreachable=0 failed=0
```

```
skipped=0 rescued=0 ignored=0
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-
```

```
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
```

```
↳ vi host_vars/srl1.yml
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-
```

```
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
```

```
↳ ansible-playbook -i inventory.yml playbook.yml
```

```
PLAY [Configure SR Linux router interfaces]
```

```
*****
```

```
*****
```

```
*****
```

```
TASK [Apply interface configuration via JSON-RPC]
```

```
*****
```

```
*****
```

```
*****
```

```
ok: [srl1]
```

```
ok: [srl2]
```

```
TASK [Show configuration result]
```

```
*****
```

```
*****
```

```
*****
```

```
ok: [srl1] => {
```

```
  "msg": "srl1: Configuration applied successfully"
```

```
}
```

```
ok: [srl2] => {
```

```
  "msg": "srl2: Configuration applied successfully"
```

```
}
```

```
TASK [Verify interfaces are configured]
```

```
*****
```

```
*****
```

```
*****
```

```
ok: [srl1]
```

```
ok: [srl2]
```

```
TASK [Display configured interfaces]
```

```
*****
```

```
*****
```

```
*****
```

```
ok: [srl2] => {
```

```
  "msg": "srl2 interfaces: [{ 'interface': [{ 'name': 'ethernet-1/1.0',  
'oper-state': 'up', 'index': '2'}, { 'name': 'ethernet-1/2.0', 'oper-state':  
'up', 'index': '3'} ] } ]"
```

```
}
```

```
ok: [srl1] => {
```

```
  "msg": "srl1 interfaces: [{ 'interface': [{ 'name': 'ethernet-1/1.0',  
'oper-state': 'up', 'index': '2'}, { 'name': 'ethernet-1/2.0', 'oper-state':  
'up', 'index': '3'}, { 'name': 'lo0.0', 'oper-state': 'up', 'index':  
'4'} ] } ]"
```

```
}
```

```
PLAY RECAP
```

```
*****
```

```
*****
```

```
*****
```

```
srl1                                : ok=4    changed=0    unreachable=0    failed=0
```

```
skipped=0    rescued=0    ignored=0
```

```
srl2                                : ok=4    changed=0    unreachable=0    failed=0
```

```
skipped=0    rescued=0    ignored=0
```

2. Verify the loopback is configured:

```
docker exec -it clab-ip-fundamentals-srl1 sr_cli -c "show interface lo0"
```

```
docker exec -it clab-ip-fundamentals-srl1 sr_cli -c "show interface lo0"
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
└─> docker exec -it clab-ip-fundamentals-srl1 sr_cli -c "show interface
lo0"
=====
=====
=====
lo0 is up, speed None, type None
  lo0.0 is up
    Network-instances:
      * Name: default (default)
        Encapsulation   : null
        Type            : None
        IPv4 addr       : 10.10.10.1/32 (static, preferred, primary)
-----
-----
-----
=====
=====
=====
Parsing error: Unknown token 'commit'. Options are ['!', '#', '/', 'acl',
'back', 'bash', 'bfd', 'cls', 'date', 'diff', 'echo', 'enter',
'environment', 'exit', 'file', 'help', 'history', 'info', 'interface',
'list',
'monitor', 'network-instance', 'passwd', 'ping', 'ping6', 'platform', 'pwc',
'qos', 'quit', 'references', 'routing-policy', 'save', 'show', 'source',
'ssh', 'system', 'tcptraceroute', 'tech-support', 'tools', 'trac
eroute', 'traceroute6', 'tree', 'tunnel', 'tunnel-interface', 'watch', '{}']
```

3. Check the routing table to see the new /32 route:

```
docker exec -it clab-ip-fundamentals-srl1 sr_cli -c "show network-
instance default route-table ipv4-unicast summary"
```

```
docker exec -it clab-ip-fundamentals-srl1 sr_cli -c "show network-instance
default route-table ipv4-unicast summary"`
```

```
[x]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
```

```
└─> docker exec -it clab-ip-fundamentals-srl1 sr_cli -c "show network-  
instance default route-table ipv4-unicast summary"
```

```
-----  
-----  
-----
```

```
IPv4 unicast route table of network instance default
```

```
-----  
-----  
-----
```

```
+-----+-----+-----+-----+  
+-----+-----+-----+-----+  
-----+-----+-----+-----+
```

Prefix			ID	Route Type	Route Owner	
Active	Origin	Metric	Pref	Next-hop (Type)	Next-	
hop	Backup Next-hop	Backup Next-hop Interface				
		Network				
Interface	(Type)					
		Instance				

```
+=====+=====+=====+=====+  
+=====+=====+=====+=====+  
=====+=====+=====+=====+
```

10.1.1.0/24			2	local	net_inst_mgr	
True	default	0	0	10.1.1.1 (direct)	ethernet-1/1.0	
10.1.1.1/32			2	host	net_inst_mgr	
True	default	0	0	None (extract)	None	
10.1.1.255/32			2	host	net_inst_mgr	
True	default	0	0	None (broadcast)		
10.1.2.0/24			3	local	net_inst_mgr	
True	default	0	0	10.1.2.1 (direct)	ethernet-1/2.0	
10.1.2.1/32			3	host	net_inst_mgr	
True	default	0	0	None (extract)	None	
10.1.2.255/32			3	host	net_inst_mgr	
True	default	0	0	None (broadcast)		
10.10.10.1/32			4	host	net_inst_mgr	
True	default	0	0	None (extract)	None	

```

+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
-----
-----
-----
-----
IPv4 routes total : 7
IPv4 prefixes with active routes : 7
IPv4 prefixes with active ECMP routes: 0
-----
-----
-----
Parsing error: Unknown token 'commit'. Options are ['!', '#', '/', 'acl',
'back', 'bash', 'bfd', 'cls', 'date', 'diff', 'echo', 'enter',
'environment', 'exit', 'file', 'help', 'history', 'info', 'interface',
'list',
'monitor', 'network-instance', 'passwd', 'ping', 'ping6', 'platform', 'pwc',
'qos', 'quit', 'references', 'routing-policy', 'save', 'show', 'source',
'ssh', 'system', 'tcptraceroute', 'tech-support', 'tools', 'trac
eroute', 'traceroute6', 'tree', 'tunnel', 'tunnel-interface', 'watch', '{}']

```

Deliverables

- Your modified `ansible/host_vars/srl1.yml`
- Output of `show interface lo0`

Exercise 4: Break/Fix -- Interface Down

Objective: Diagnose and fix an admin-disabled interface.

Setup (break it)

```
docker exec -it clab-ip-fundamentals-srl1 sr_cli
```

Inside SR Linux:

```
enter candidate
set / interface ethernet-1/1 admin-state disable
commit now
```

```
[x]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
└─> docker exec -it clab-ip-fundamentals-srl1 sr_cli
Using configuration file(s): []
Welcome to the srlinux CLI.
Type 'help' (and press <ENTER>) if you need any help using this.
--{ + running }--[ ]--
A:srl1# enter candidate
--{ + candidate shared default }--[ ]--
A:srl1# set / interface ethernet-1/1 admin-state disable
--{ +* candidate shared default }--[ ]--
A:srl1# commit now
All changes have been committed. Leaving candidate mode.
--{ + running }--[ ]--
A:srl1#
```

Symptom

```
docker exec clab-ip-fundamentals-host1 ping -c 3 10.1.1.1
# FAILS -- no response
```

Your Task

1. Diagnose: Which interface is down? Use `show interface brief` on srl1.
2. Fix: Re-enable the interface and commit.
3. Verify: Confirm ping works again.

Deliverables

Document the diagnostic commands you used and the fix you applied.

Diagnosis:

```
A:srl1# show interface brief
```

Interface	Admin	Oper	Speed	Type	
ethernet-1/1	disable	down			<--
PROBLEM					
ethernet-1/2	enable	up	25G	ethernet	

The `Admin` column shows `disable` for ethernet-1/1. This means someone (us) explicitly disabled the interface. The `Oper` state is `down` because the admin state overrides.

Fix:

```
A:srl1# enter candidate
```

```
set / interface ethernet-1/1 admin-state enable
```

```
commit now
```

```
[x]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-
```

```
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
```

```
↳ docker exec -it clab-ip-fundamentals-srl1 sr_cli
```

```
Using configuration file(s): []
```

```
Welcome to the srlinux CLI.
```

```
Type 'help' (and press <ENTER>) if you need any help using this.
```

```
--{ + running }--[ ]--
```

```
A:srl1# enter candidate
```

```
--{ + candidate shared default }--[ ]--
```

```
A:srl1# set / interface ethernet-1/1 admin-state enable
```

```
--{ +* candidate shared default }--[ ]--
```

```
A:srl1# commit now
```

```
All changes have been committed. Leaving candidate mode.
```

```
--{ + running }--[ ]--
```

Verification:

```
$ docker exec clab-ip-fundamentals-host1 ping -c 3 10.1.1.1
```

```
3 packets transmitted, 3 packets received, 0% packet loss
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/02-ip-fundamentals/ansible]  
└─> docker exec clab-ip-fundamentals-host1 ping -c 3 10.1.1.1  
PING 10.1.1.1 (10.1.1.1): 56 data bytes  
64 bytes from 10.1.1.1: seq=0 ttl=64 time=66.310 ms  
64 bytes from 10.1.1.1: seq=1 ttl=64 time=1.912 ms  
64 bytes from 10.1.1.1: seq=2 ttl=64 time=1.808 ms  
  
--- 10.1.1.1 ping statistics ---  
3 packets transmitted, 3 packets received, 0% packet loss  
round-trip min/avg/max = 1.808/23.343/66.310 ms  
  
|  
|
```

Hint

Key lesson: Always check admin-state first. An operationally down interface might just be admin-disabled, not physically broken.

Exercise 5: Break/Fix -- Subnet Mismatch

Objective: Diagnose connectivity failure caused by mismatched subnet configuration.

Setup (break it)

```
docker exec clab-ip-fundamentals-host1 ip addr del 10.1.1.2/24 dev eth1  
docker exec clab-ip-fundamentals-host1 ip addr add 10.1.1.200/30 dev eth1
```

Symptom

```
docker exec clab-ip-fundamentals-host1 ping -c 3 10.1.1.1  
# FAILS -- "Network is unreachable"
```

```
[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
└─> docker exec clab-ip-fundamentals-host1 ip addr del 10.1.1.2/24 dev eth1
[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
└─> docker exec clab-ip-fundamentals-host1 ip addr add 10.1.1.200/30 dev
eth1
[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
└─> docker exec clab-ip-fundamentals-host1 ping -c 3 10.1.1.1
PING 10.1.1.1 (10.1.1.1): 56 data bytes
^C
```

Your Task

1. Check host1's IP configuration:

```
docker exec clab-ip-fundamentals-host1 ip addr show eth1
```

```
[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
└─> docker exec clab-ip-fundamentals-host1 ip addr show eth1
208: eth1@if209: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 9500 qdisc
noqueue state UP
    link/ether aa:c1:ab:b0:39:ea brd ff:ff:ff:ff:ff:ff
    inet 10.1.1.200/30 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::a8c1:abff:feb0:39ea/64 scope link
        valid_lft forever preferred_lft forever

```

1. Calculate the /30 subnet for 10.1.1.200:
 - Network: 10.1.1.200
 - Broadcast: 10.1.1.203
 - Usable range: 10.1.1.201 - 10.1.1.202
2. Is srl1 (10.1.1.1) within that range? Why or why not?
 - Because it is not in the usable range 10.1.1.201 - 10.1.1.202

3. Fix by restoring the correct address:

```
docker exec clab-ip-fundamentals-host1 ip addr del 10.1.1.200/30 dev eth1
docker exec clab-ip-fundamentals-host1 ip addr add 10.1.1.2/24 dev eth1
```

4. Verify ping works again.

```
[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
└─> docker exec clab-ip-fundamentals-host1 ip addr del 10.1.1.200/30 dev
eth1
[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
└─> docker exec clab-ip-fundamentals-host1 ip addr add 10.1.1.2/24 dev eth1
[*]--[kc-debian13-test00-Klon]--[kubecraft/github_rbeckesch_DrewElliot-
kubecraft/lessons/clab/02-ip-fundamentals/ansible]
└─> docker exec clab-ip-fundamentals-host1 ping -c 3 10.1.1.1
PING 10.1.1.1 (10.1.1.1): 56 data bytes
64 bytes from 10.1.1.1: seq=0 ttl=64 time=3.347 ms
64 bytes from 10.1.1.1: seq=1 ttl=64 time=2.355 ms
64 bytes from 10.1.1.1: seq=2 ttl=64 time=2.155 ms
--- 10.1.1.1 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 2.155/2.619/3.347 ms
```

Deliverables

Explain why a /30 mask at .200 prevents reaching .1, and how subnet masks determine what is "local."

- /30 is equal to 255.255.25.252 is equal to 11111111.11111111.11111111.11111100
- The netmasks 1 - ones determine the "network part" of the address, so there are only four addresses left to be used in the network for the single hosts.

✓ Check

Key lesson: Subnet masks determine what's "local" (reachable via Layer 2) and what's "remote" (needs a router). Both endpoints must agree on the subnet for direct communication to work.

Exercise 6: Break/Fix -- Missing Gateway

Objective: Understand why a default route is needed for cross-subnet communication.

Setup (break it)

```
docker exec clab-ip-fundamentals-host1 ip route del default
```

Symptom

```
# This works (same subnet)
docker exec clab-ip-fundamentals-host1 ping -c 3 10.1.1.1
[
# This fails (different subnet)
docker exec clab-ip-fundamentals-host1 ping -c 3 10.1.2.1
# "Network is unreachable"
```

Your Task

1. Check the routing table:

```
docker exec clab-ip-fundamentals-host1 ip route show
```

2. Notice: only the `10.1.1.0/24` route exists (directly connected). No default route.
3. Explain: Why does 10.1.1.1 work but 10.1.2.1 doesn't?
4. Fix by restoring the default route:

```
docker exec clab-ip-fundamentals-host1 ip route add default via 10.1.1.1
dev eth1
```

5. Verify: ping 10.1.2.1 now works (srl1 has that subnet directly connected).

Deliverables

Explain the difference between local and remote subnets, and why gateways are needed.

The local subnet is directly addressed via the local available network link (eg NIC). If a request addresses a remote subnet, the local system can not reach the remote subnet directly via the local link, so there must be alternative ways to reach this kind of addresses (also miscellaneous internet addresses) via defined routes. So there may be routes to reach the remote subnet via a concrete gateway/router defined in the routing table, to reach this concrete subnet, but it also a often seen usecase, that the local system has a generic default gateway, that does the whole routing to other subnets (and also "the internet"), so the local host needs a default route only.

Cleanup

After completing all exercises:

```
# Destroy the lab
containerlab destroy -t topology/lab.clab.yml --cleanup
|
# Verify
docker ps | grep clab
```

Validation

Run the automated tests:

```
pytest tests/ -v
```

```
[*]-[kc-debian13-test00-Klon]-[/kubecraft/github_rbeckesch_DrewElliot-  
kubecraft/lessons/clab/02-ip-fundamentals]
```

```
└─> pytest tests/ -v
```

```
=====
```

```
===== test session starts
```

```
=====
```

```
=====
```

```
platform linux -- Python 3.13.5, pytest-8.3.5, pluggy-1.5.0 --
```

```
/usr/bin/python3
```

```
cachedir: .pytest_cache
```

```
rootdir: /kubecraft/github_rbeckesch_DrewElliot-kubecraft
```

```
configfile: pyproject.toml
```

```
plugins: anyio-4.8.0
```

```
collected 25 items
```

```
tests/test_ip_fundamentals.py::TestEnvironment::test_containerlab_installed
```

```
PASSED
```

```
[ 4%]
```

```
tests/test_ip_fundamentals.py::TestEnvironment::test_docker_available PASSED
```

```
[ 8%]
```

```
tests/test_ip_fundamentals.py::TestEnvironment::test_ansible_installed
```

```
PASSED
```

```
[ 12%]
```

```
tests/test_ip_fundamentals.py::TestEnvironment::test_topology_file_exists
```

```
PASSED
```

```
[ 16%]
```

```
tests/test_ip_fundamentals.py::TestEnvironment::test_topology_file_valid_yam
```

```
l PASSED
```

```
[ 20%]
```

```
tests/test_ip_fundamentals.py::TestTopologyStructure::test_has_name PASSED
```

```
[ 24%]
```

```
tests/test_ip_fundamentals.py::TestTopologyStructure::test_has_four_nodes
```

```
PASSED
```

```
[ 28%]
```

```
tests/test_ip_fundamentals.py::TestTopologyStructure::test_has_srl_routers
```

```
PASSED
```

```
[ 32%]
```

```
tests/test_ip_fundamentals.py::TestTopologyStructure::test_has_linux_hosts
```

```
PASSED
```

```
[ 36%]
```

```
tests/test_ip_fundamentals.py::TestTopologyStructure::test_has_three_links
```

```
PASSED
```

```
[ 40%]
```

```
tests/test_ip_fundamentals.py::TestTopologyStructure::test_no_latest_tags
```

```
PASSED
```

```
[ 44%]
```

```
tests/test_ip_fundamentals.py::TestTopologyStructure::test_hosts_have_exec
```

PASSED

[48%]

tests/test_ip_fundamentals.py::TestTopologyStructure::test_routers_have_mgmt_ips PASSED

[52%]

tests/test_ip_fundamentals.py::TestAnsibleStructure::test_inventory_exists

PASSED

[56%]

tests/test_ip_fundamentals.py::TestAnsibleStructure::test_playbook_exists

PASSED

[60%]

tests/test_ip_fundamentals.py::TestAnsibleStructure::test_template_exists

PASSED

[64%]

tests/test_ip_fundamentals.py::TestAnsibleStructure::test_host_vars_exist

PASSED

[68%]

tests/test_ip_fundamentals.py::TestAnsibleStructure::test_inventory_valid_yaml PASSED

[72%]

tests/test_ip_fundamentals.py::TestAnsibleStructure::test_host_vars_have_interfaces PASSED

[76%]

tests/test_ip_fundamentals.py::TestAnsibleStructure::test_template_has_jinja2_syntax PASSED

[80%]

tests/test_ip_fundamentals.py::TestLabDeployment::test_lab_deploys_successfully PASSED

[84%]

tests/test_ip_fundamentals.py::TestLabDeployment::test_all_containers_running PASSED

[88%]

tests/test_ip_fundamentals.py::TestConnectivity::test_host1_pings_srl1

PASSED

[92%]

tests/test_ip_fundamentals.py::TestConnectivity::test_host2_pings_srl2

PASSED

[96%]

tests/test_ip_fundamentals.py::TestConnectivity::test_srl1_pings_srl2 PASSED

[100%]

=====

===== 25 passed in 249.97s (0:04:09) =====

=====

=====

Completion Checklist

- ✓ Exercise 1: Deployed lab, applied config, verified adjacent pings work
- ✓ Exercise 2: Traced Jinja2 template for srl2, wrote generated commands
- ✓ Exercise 3: Added loopback interface via Ansible
- ✓ Exercise 4: Diagnosed and fixed admin-disabled interface
- ✓ Exercise 5: Diagnosed and fixed subnet mismatch
- ✓ Exercise 6: Diagnosed and fixed missing default route

Next Steps

Commit your exercises to your fork and proceed to Lesson 3: Routing Basics.